That's a great idea. Tables make it much easier to compare syntax and behavior at a glance. Here are the updated notes with a focus on structured tables.

---

1. Input/Output (I/O)

Java uses the Scanner class for input and System.out for output.

Feature	Method	Description
Basic Output	print()	Outputs text on the same line.
New Line Output	println()	Outputs text and moves the cursor to the next line.
Formatted Output	printf()	Uses placeholders like %d (int) or %f (double).
String Input	next() / nextLine()	next() reads one word; nextLine() reads the full line.
Numeric Input	nextInt() / nextDouble()	Reads the next available primitive number.

---

2. Operators in Java

Operators are the building blocks of logic.

Comparison & Logical Operators

Category	Operator	Meaning	Example
Relational	== , !=	Equal to, Not equal to	x == 10
	>, <	Greater than, Less	x > 5

<table>
<tr><td></td><td></td><td>than</td><td></td></tr>
<tr><td>Logical</td><td>&&</td><td>AND: True if both sides are true</td><td>(x > 0 && x < 10)</td></tr>
<tr><td></td><td>||</td><td>OR: True if at least one side is true</td><td>(x == 1 || x == 2)</td></tr>
<tr><td></td><td>!</td><td>NOT: Reverses the boolean value</td><td>!(x > 5)</td></tr>
</table>

Arithmetic & Assignment

Operator	**Name**	**Behavior**
%	Modulus	Returns the remainder (e.g., 10 % 3 is 1).
++ / --	Increment/Decrement	Increases or decreases value by 1.
+=, -=, *=	Compound Assignment	x += 5 is shorthand for x = x + 5.

---

3. Control Flow (Decisions & Loops)

Control flow dictates the path your program takes.

Decision Making

Structure	**Best Used For...**
If-Else	Checking ranges or complex boolean logic (e.g., x > 10 && y < 20).
Switch	Checking a single variable against many constant values (Integers, Strings, Enums).

Ternary (? :)	Short, one-line if-else assignments: String result = (age > 18) ? "Adult" : "Minor";

Loops (Repetitive Logic)

Loop Type	Execution Logic	Minimum Iterations
For	Initialization → Condition → Body → Update	0
While	Condition → Body	0
Do-While	Body → Condition	**1**
For-Each	Iterates through every element in an array/list	0

---

4. Critical Mistakes to Avoid

Don't fall into these common "traps" that even intermediate developers sometimes hit:

Mistake	Why it's a problem	Correct Way
str1 == str2	Compares memory addresses, not text.	str1.equals(str2)
Floating Point ==	Decimals (double) have tiny precision errors.	Math.abs(a - b) < 0.00001
The "Scanner Bug"	nextInt() leaves a newline \n in the buffer.	Follow nextInt() with a dummy nextLine().
Infinite While Loop	Forgetting to update the variable in the condition.	Always check your i++ or loop exit logic.
Array Out of Bounds	Trying to access index N in	Last index is always size - 1.

	an array of size N.	

---

5. Summary Cheat Sheet

- **Strings:** Always use .equals().
- **Integer Division:** 5 / 2 results in 2. Use 5 / 2.0 to get 2.5.
- **Switch Statement:** Don't forget the break; keyword unless you want "fall-through" behavior.
- **Operators:** && and || are "short-circuiting"—if the first part of an && is false, Java won't even check the second part.

5. Summary of Jump Statements

Inside loops, you can use these two keywords to control behavior:

- `break`: Exits the loop entirely.
- `continue`: Skips the rest of the current iteration and jumps to the next one.